Jev in a Multi-Step Workflow: Predicting Failures Is Not Knowing Which Reviews Pay

Thejas P Rao
thejasrao262003@gmail.com

Tushar M
tusharm3040@gmail.com

Dr Prajwala T R
PES University, Bengaluru, India
prajwalatr@pes.edu

***Abstract*—Many systems chain an AI model's typed decisions into a workflow and use the model's own confidence to decide which cases a human should check. We test that practice with Jev, TypeSafe's typed decision model, whose three primitives (a categorical `choice`, a binary `noul` and an ordinal `score`) we chain into a bug-triage workflow. After exploratory work on 442 Mozilla and Linux kernel bugs, we ran pre-registered tests on 500 new bugs, four further models and an independent Kubernetes workflow. A failure is a disagreement between the workflow's action and a target action built from reference labels. A frozen estimator that reads only the workflow's decisions ranks failures better than final-step confidence (AUROC 0.76 vs. 0.55 on the new bugs), for all five models, and most of its signal is how often the chosen action was wrong before. The Kubernetes replication agreed (AUROC 0.78 vs. 0.63), as did three of four further models, including the only other one competent at every step. These error rates do not transfer across label mixes, and a label-free correction that helped on the bug trackers made ranking worse on Kubernetes. Better ranking did not reduce cost: under the pre-registered costs, no 10% review policy beat no review. The savings sit in wrongly accepted cases that none of the tested signals could pick out.**

***Index Terms*—uncertainty quantification, calibration, structured outputs, decision workflows, selective prediction, escalation, label shift, human review, LLM evaluation, bug triage**

I. Introduction

Typed decision models return an answer in a fixed format, such as a label, a yes/no or a rating, together with a probability or a confidence. The answer can be well formed and still wrong. Chain several of these answers into a workflow and a mistake at one step becomes the input to the next. Practitioners often assume that the confidence each step reports can be combined into one workflow-level signal that says when a person should look at a case.

Jev, TypeSafe's typed decision model, has been benchmarked on single tasks [1]–[3], and uncertainty along multi-step agent trajectories has been studied separately [4]. Our question is operational. When a workflow chains several typed decisions, which signals identify the cases where it fails, and do those signals find the costly errors that justify a human review? The two parts can come apart. A signal can rank failures well and still be of little use for choosing what to review.

Jev's primitives also report uncertainty in different forms. `choice` returns a probability for each label plus a scalar confidence, `score` returns an ordinal distribution plus a scalar confidence, and `noul` returns a single probability with no confidence field. TypeSafe describes its confidence as one reasonable summary of a distribution and advises setting thresholds on one's own data [5]. So we do not test a calibration claim. We measure how these numbers behave as escalation signals once the primitives are chained.

We ask three questions. RQ1: which signals rank workflow failures, the model's confidence or what the workflow decided? RQ2: do those signals stay useful when the label distribution shifts between development and deployment? RQ3: do signals that rank failures also find the reviews that reduce cost?

The contribution is methodological. We measure uncertainty at the level of a workflow, where failure is defined on the action taken rather than on any single answer. We separate ranking failures from reducing cost by scoring a fixed review budget against a frozen cost matrix and an oracle bound. And we pre-registered every confirmatory test, froze the estimators by hash before their data existed, and replicated the main result across five models and a second, chronologically split workflow.

In short, the workflow's decisions predict its failures much better than its confidence does, and the gap holds across models and workflows. Most of that signal is the error rate of the chosen action. Those error rates shift with the label mix. And none of the tested signals made review pay under our costs.

II. Related work

Jev and typed decisions. Deußer et al. benchmarked Jev zero-shot on 37 datasets and found well-calibrated `choice` probabilities, while binary probabilities ranked well but sat poorly against a 0.5 threshold [1]. A calibration audit found that calibration changed with the question form and that one fitted temperature removed most miscalibration [2], and a study of political-science replications found Jev's calibration advantage inconsistent [3]. Open instruction-tuned models can produce Jev-like decisions from answer-label logits [6], which is the readout we use for the general models. All of this work evaluates single requests.

Calibration, selection and deferral. Post-hoc calibration [7], [8], selective prediction [9], [10] and learning to defer [11], [12] all concern one decision. Bansal et al. show that the most accurate model need not make the best human–AI team [13], and Park et al. argue that error risk and review value are different quantities under a review budget [14]. Our cost result is a case of the second point in a dependent workflow.

Label shift and composition. Posteriors can be adjusted to new class priors [15], and the priors can be estimated from a black-box predictor's confusion matrix (BBSE) [16]. Product rules for combining classifiers assume independent, comparable scores [17]. Conformal methods have been composed over pipeline stages [18], and step-level calibration does not guarantee trajectory-level calibration for agents [4]. LLM raters compress ordinal scales toward the middle [19].

Cascades and software reliability. Cascades route a single query through model tiers and decide when to defer [20]–[22]. Component-based reliability models treat error propagation between components as an architectural property that decides where detection and recovery belong [23], and bug severity prediction from report text is a long-studied triage task [24]. Our workflow is a small instance of the same propagation problem, with human escalation as the recovery step.

We found no prior work that tests, on real labels and held-out data, which signals predict failure in a chain of structurally different typed decisions and whether they find the reviews that pay.

III. Methodology

Data and workflow. Figure 1 shows the pipeline. We use 202 Mozilla and 240 Linux kernel networking and storage bugs for development, 500 newer bugs (250 per tracker, natural label mix) as a held-out set, and 1,082 Kubernetes issues for the replication. Texts are privacy-scrubbed and truncated to 2,000 characters. Node 1 (`choice`) routes a report to a subsystem, node 2 (`noul`) asks whether it is a regression, and node 3 (`score`) rates severity on three levels. Each node sees the evidence plus the earlier decisions. Every node is called three times, and the answers are reduced to one decision by modal vote (for `noul`, the mean probability is thresholded at 0.5). All Jev calls resolved to `jev-1.13.0`.

Action, target and failure. The rule F maps an unclear c_2 to *request evidence*, $c_3 = 2$ to *escalate*, $c_3 = 1$ with $c_2 = 1$ to *defer*, and anything else to *accept*. The target is the same rule on the reference labels, $a^* = F(c_2^*, c_3^*)$, and a failure is $y = \mathbf{1}[a \neq a^*]$. F is our construct, so every failure result is conditional on it.

Failure signals. We compare three kinds of signal: the model's final-step confidence, alone or composed across nodes; the action-rule score $r(a) = (e(a) + \bar{r})/(n(a) + 1)$, the smoothed error rate of the chosen action on development data; and a logistic estimator on the decisions (E_{dec}), fitted on development data and frozen.

Evaluation. AUROCs use the Mann–Whitney estimator with 95% bootstrap intervals over bugs ($B = 2{,}000$). Confirmatory tests are one-sided paired bootstrap tests with Holm correction. Costs come from a matrix frozen in advance: a wrong *accept* costs 6–10, any other mismatch 1, and each escalation 1 more for the reviewer's time.

Novelty. Table I lists what is new in the design and what each element lets us conclude.

TABLE I
Methodological elements and what each allows us to conclude.

Element	What it allows
Workflow-level failure	Failure is defined on the action taken, so every step-level signal is tested as a predictor of it
Ranking vs. cost	A fixed 10% review budget, a frozen cost matrix and an oracle bound separate "ranks failures" from "reduces cost"
Shift by design	Development and deployment label mixes differ, which exposes the label-prior dependence of error rates
Pre-registration	Hash-frozen estimators and dated amendments separate confirmatory from exploratory claims
Replication	Five models behind one interface and an independent Kubernetes workflow

TABLE II
Node agreement with reference labels on the development bugs (Wilson 95% intervals). Baseline: the most frequent label.

	Mozilla		Kernel	
Node	Agreement	Base	Agreement	Base
`choice`	0.89 [0.83, 0.92]	0.59	0.975 [0.95, 0.99]	0.50
`noul`	0.80 [0.74, 0.85]	0.74	0.725 [0.67, 0.78]	0.81
`score`	0.53 [0.46, 0.60]	0.40	0.44 [0.38, 0.50]	0.33

IV. Exploratory findings on the development bugs

The three primitives' confidences behave very differently. `choice` confidence is near 1.0 for most bugs (85% of Mozilla and 96% of kernel bugs are at ≥ 0.90), yet it still discriminates its own errors (AUROC 0.90 on Mozilla, 0.98 on the kernel). `noul`'s confidence-like value carries no usable information (AUROC 0.53 and 0.41), and `score`'s confidence is about the same at every true level (AUROC 0.58 and 0.57). Fixed thresholds tuned on constructed data did not transfer: on real text a `choice` trigger at 0.90 fired on 85% and 96% of bugs, and a product cut escalated 88% and 78% against targets of 41% and 42%.

The answers themselves are uneven (Table II). `choice` routes well above baseline on both trackers. `noul`'s errors change direction: on Mozilla it mostly misses regressions (23 misses vs. 17 false alarms), while on the kernel it over-flags them (63 false alarms vs. 3 misses). `score` compresses severity toward the middle level (Fig. 2). Most minor bugs are rated higher than their label (79% on Mozilla, 75% on the kernel), and many serious bugs lower (43% and 56%). Using Mozilla's own severity definitions moved agreement only from 0.53 to 0.56, so the compression does not come from our mapping.

Composing confidences did not help. A naive comparison looks favourable to decisions: on Mozilla, the decision-based action beat a confidence-product rule by 0.17 [0.08, 0.27] in the probability of the right action (exact McNemar $p = 0.0004$). But the confidence rule never reads the decisions,

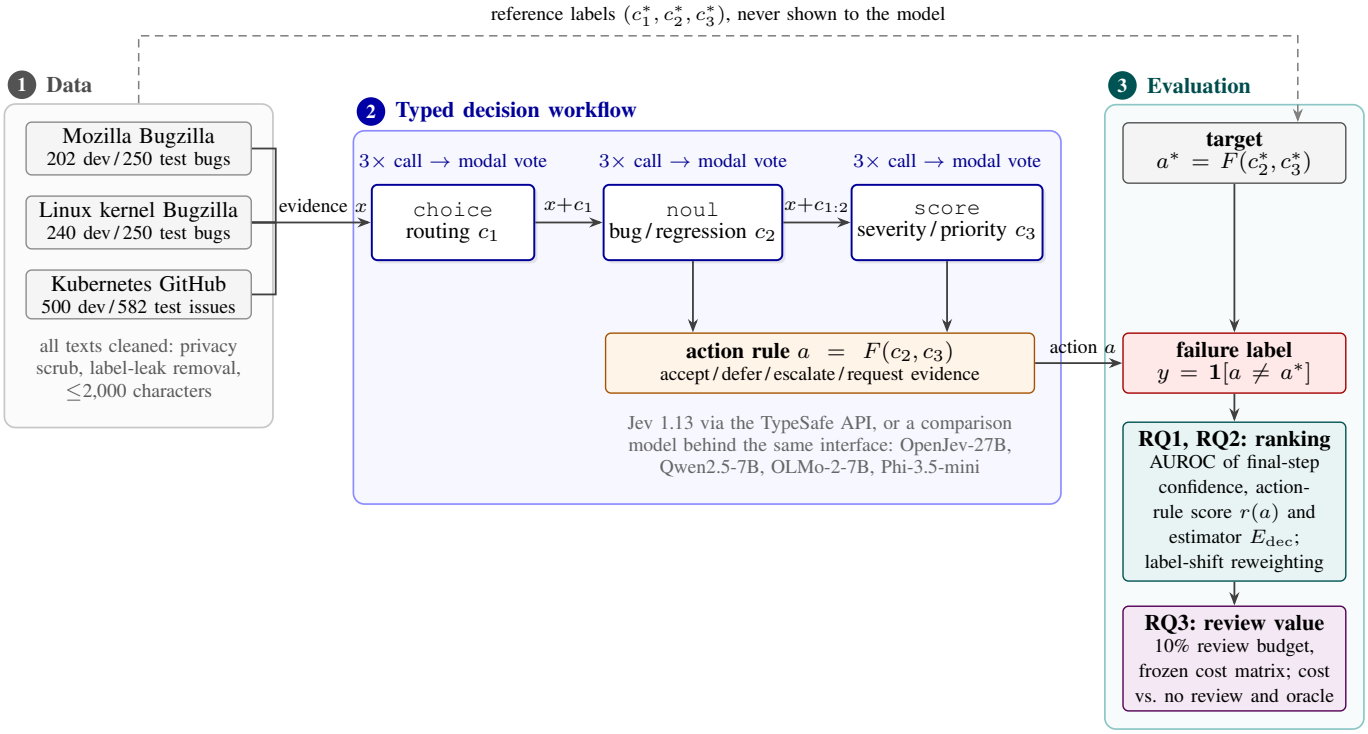


Fig. 1. Study architecture. Reports from three public trackers are cleaned and stripped of label leaks, then pass through three typed decisions, each called three times and reduced by modal vote. An action rule F maps the second and third decisions to an action. Reference labels, never shown to the model, give the target a^* through the same rule, and a failure is $a \neq a^*$.

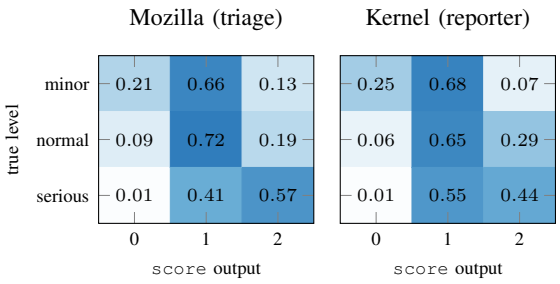


Fig. 2. Row-normalized confusion of `score` against reference severity. On both trackers the middle column absorbs mass from both extremes.

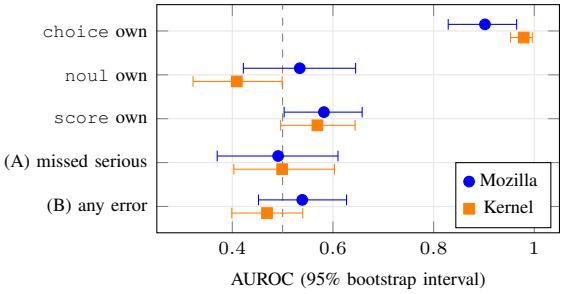


Fig. 3. Confidence AUROCs on the development bugs; dashed line: chance. Only `choice`'s confidence discriminates its own errors, and composed confidence is at chance for both escalation targets, (A) missed serious bugs and (B) any wrong action.

while the target escalates on the severity decision, so that comparison is confounded. The fairer test starts from the decision-based action and adds escalations ranked by low composed confidence. Among bugs the rule did not escalate, composed confidence ranked missed serious bugs at chance (AUROC 0.49 and 0.50), and across all bugs it ranked wrong actions at chance as well (0.54 and 0.47). At a budget of 10% of bugs, confidence-ranked overrides did no better than random selection on either tracker. Every one of these intervals covers 0.5 (Fig. 3). The reason is structural. The only informative confidence belongs to `choice`, the one node F does not read.

These observations led to the pre-registered tests below.

V. Pre-registered held-out test

We trained E_{dec} (L_2-regularized logistic regression on the `score` level, the `noul` decision and the action) on the 442 development bugs, froze it by hash, and only then collected the 500 held-out bugs (4,500 Jev calls). We pre-registered two hypotheses: E_{dec} ranks wrong actions better than low final-step confidence (HR1), and above chance on each tracker (HR2).

Both were supported. E_{dec} reached AUROC 0.763 against 0.549 for confidence, a difference of $+0.214$ with a one-sided lower bound of $+0.158$. Per tracker it reached 0.811 on Mozilla and 0.697 on the kernel, and the gap over confidence was $+0.279$ $[+0.191, +0.374]$ on Mozilla and a weaker

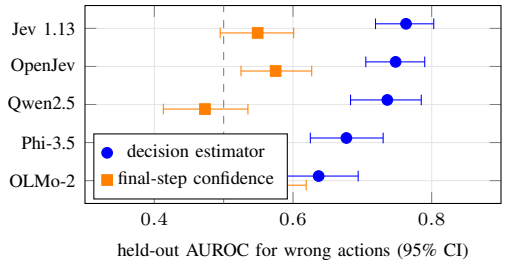


Fig. 4. For all five models, the decision estimator ranks wrong workflow actions better than the model's final-step confidence on the 500 held-out bugs.

$+0.124$ [$+0.023$, $+0.225$] on the kernel. On the 70 Mozilla bugs filed in the 12 months before sampling, which are least likely to appear in model training data, E_{dec} reached 0.847 against 0.519. The result held when every development bug linked to a held-out bug was removed and the estimator refitted (0.772 against 0.551), and an independent script that shares no code with the analysis reproduced every point estimate. The estimator's predicted failure rate matches the observed rate overall, but it outputs only 7–8 distinct risk levels (ECE 0.18–0.20), so it ranks better than it calibrates.

VI. Cross-model replication

To see whether the result is specific to Jev, we ran the same bugs, questions and adapter through four more models: OpenJev-27B [25], an independently trained Jev-style model, and three small general instruct models, Qwen2.5-7B [26], OLMo-2-7B [27] and Phi-3.5-mini [28]. Jev is used zero-shot, so the fair comparison is other models used the same way, through the same questions and TypeSafe's confidence formulas. We picked small open models because they are cheap, reproducible and make more mistakes, which gives the failure predictors more to find. The protocol had three amendments: OpenJev was added before any of its outputs existed, OLMo-2 needed a newer `transformers` release, and a readout bug found after interim results led us to re-run all three general models. The conclusions did not change.

All 8 pre-registered tests were supported (Fig. 4, Table III). Decision estimators reached AUROC 0.64–0.76, final-step confidence 0.40–0.58. The weakest case was OLMo-2 (0.637 vs. 0.551), which passed at the last Holm step. Jev's frozen estimator, never refitted, reached 0.76 [0.72, 0.80] on OpenJev's outputs, as well as OpenJev's own estimator did.

Most of this signal is simply which action was chosen. The action's error rate on development data matches the estimator for Jev (0.76) and OpenJev (0.75) and comes close for the others (0.58–0.70). For Jev the estimator adds $+0.000$ [-0.012, $+0.011$] beyond that rate. For Qwen2.5 and OLMo-2 it adds a little ($+0.035$ and $+0.057$). Single uncertainty signals sit at chance: routing confidence at 0.48–0.52, regression confidence at 0.45–0.56 and severity entropy at 0.40–0.55. Adding them to the decision features helps only the weakest general models ($+0.054$ for Phi-3.5, $+0.078$ for OLMo-2). Part of this is built in, since the action and the target come from the same rule F. The two model families also fail differently. Jev and OpenJev pull severity to the middle level, while the general models call 67–100% of bugs "blocking" whatever the true level, and are wrong far more often (72–80% vs. 43–47%).

That makes the rates sensitive to the label mix (RQ2). The development set was quota-sampled by severity and the held-out set was not. On a target that bypasses F (the severity answer being wrong), estimators fitted on development data ranked held-out errors far below chance, 0.28 for Jev and OpenJev. Jev's level-1 answers were 60% wrong in the development set and 20% wrong in the natural mix. Re-weighting to the natural mix fixed this for four of five models (Table IV; Jev 0.79 vs. 0.61 for confidence).

Estimating the mix without labels by BBSE [16] recovered most of that gain for Jev (0.74), OpenJev (0.78) and Qwen2.5 (0.74), but not for the right reason. The estimated mix was wrong. For the kernel it was $[0, 1, 0]$ against a true $[0.02, 0.73, 0.25]$, and the ranking improved only because the middle level happened to dominate. Under known synthetic shifts the estimate was biased toward the development mix, and for Phi-3.5, whose severity answers are nearly all "blocking", it gave no gain at all.

VII. An independent workflow: Kubernetes

We pre-registered a replication on issue triage in the Kubernetes GitHub repository before collecting any of its data. It differs in platform, labelling process, routing (network, storage, node) and questions (bug or not, priority). The split is chronological: 500 issues from 2015–2019 for development and the 582 latest eligible issues from 2021–2026 for testing, with linked issues removed and label leaks stripped from the text.

For Jev the action rule again beat confidence, 0.776 against 0.628, a paired difference of $+0.148$ [$+0.091$, $+0.207$]. As before, the decision estimator added little ($+0.004$ to $+0.017$), and the contrast held for the any-node target, within SIG Node, and on issues triaged within 30 days. Because the two periods also differ in issue format and label mix, this is a combined test of temporal and distribution shift. A second pre-registration ran the four other models (Table V). By its fixed rule, at least three of four, the finding holds. The detail matters, though. Only Jev and OpenJev-27B beat chance on every step. The three general models handled routing and the bug question but failed on priority, so their workflows were wrong most of the time (wrong-action rates up to 95%), and the rule "won" largely by flagging the one action each model nearly always chose. Phi-3.5 escalated 580 of the 582 test issues, which left the rule nothing to rank.

The label-free correction did not carry over. Re-weighting a deliberately mismatched fit by BBSE lowered its AUROC from 0.699 to 0.348. BBSE again estimated the mix as entirely middle level, $[0, 1, 0]$, while the true mix was $[0.69, 0.26, 0.05]$, so the weighting pushed the fit the wrong way. This supports our reading of the earlier gains as a coincidence of a dominant middle level.

TABLE III

PER-MODEL RESULTS ON THE 500 HELD-OUT BUGS. AUROC COLUMNS RANK WRONG WORKFLOW ACTIONS (95% BOOTSTRAP CI): THE DECISION ESTIMATOR FITTED ON EACH MODEL'S OWN DEVELOPMENT OUTPUTS, FINAL-STEP CONFIDENCE, THE ACTION-RULE SCORE AND JEV'S FROZEN ESTIMATOR APPLIED UNCHANGED. COST COLUMNS ARE FROM THE PRE-REGISTERED REVIEW TEST (SECTION VIII); THE ORACLE IS AN UPPER BOUND THAT MUST SPEND ALL 50 REVIEWS.

			Held-out AUROC for wrong actions				Cost at 10% review budget			
Model	Type	Fail rate	Decision est.	Final-step conf.	Action rule	Jev est.	No review	Best review arm	Oracle	Wrong accepts
Jev	TypeSafe, hosted	0.43	0.76 [0.72, 0.80]	0.55 [0.49, 0.60]	0.76	—	658	681 (conf. product)	370	52%
OpenJev	Jev-style, 27B	0.47	0.75 [0.70, 0.79]	0.57 [0.53, 0.63]	0.75	0.76	660	674 (confidence)	441	40%
Qwen2.5	general, 7B	0.74	0.74 [0.68, 0.79]	0.47 [0.41, 0.54]	0.70	0.74	903	910 (estimator)	856	14%
Phi-3.5	general, 3.8B	0.72	0.68 [0.62, 0.73]	0.40 [0.34, 0.46]	0.69	0.70	831	890 (confidence)	890	4%
OLMo-2	general, 7B	0.80	0.64 [0.58, 0.69]	0.55 [0.48, 0.62]	0.58	0.64	893	916 (all arms tie)	916	1%

TABLE IV

EXPLORATORY: HELD-OUT AUROC OF THE DECISION ESTIMATOR ON TWO TARGETS THAT BYPASS F, FITTED ON THE QUOTA-SAMPLED DEVELOPMENT BUGS ("DEV FIT") OR RE-WEIGHTED TO THE NATURAL LABEL MIX, AGAINST FINAL-STEP CONFIDENCE. BOLD: RE-WEIGHTED ESTIMATOR ABOVE CONFIDENCE.

	Severity wrong			Any node wrong		
Model	dev fit	re-weighted	conf.	dev fit	re-weighted	conf.
Jev	0.28	**0.79**	0.61	0.56	**0.76**	0.56
OpenJev	0.28	**0.79**	0.63	0.62	**0.81**	0.56
Qwen2.5	0.62	**0.74**	0.46	0.76	**0.73**	0.40
Phi-3.5	0.31	**0.67**	0.41	0.67	**0.77**	0.29
OLMo-2	0.50	**0.62**	0.57	0.42	0.42	0.49

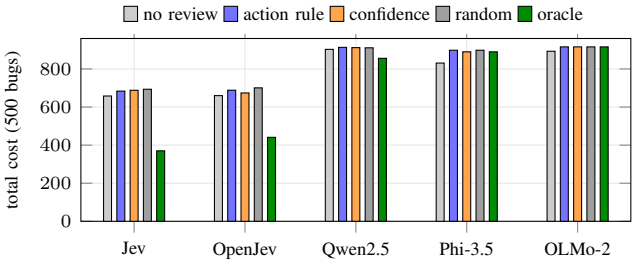


Fig. 5. Total cost at a 10% review budget under the pre-registered cost matrix. No feasible strategy beats no review. A perfect reviewer (oracle) would cut Jev's cost by 44% and OpenJev's by 33%.

TABLE V

KUBERNETES TEST SET: AUROC FOR WRONG ACTIONS OF THE ACTION-RULE SCORE AND OF FINAL-STEP CONFIDENCE, AND PRIORITY ACCURACY (CHANCE 0.33).

Model	Rule	Conf.	Priority acc.	Result
Jev 1.13	0.776	0.628	0.371	supported
OpenJev-27B	0.796	0.569	0.404	supported
Qwen2.5-7B	0.710	0.555	0.270	supported
OLMo-2-7B	0.850	0.255	0.082	supported
Phi-3.5-mini	0.519	0.674	0.046	not replicated

TABLE VI

EXPLORATORY COST SENSITIVITY: BEST FEASIBLE ARM MINUS NO REVIEW (NEGATIVE, IN BOLD: REVIEW REDUCES COST), RANKINGS FROZEN. m SCALES WRONG-ACCEPT COSTS, h IS THE HUMAN COST PER ESCALATION AND p^* THE BREAK-EVEN PRECISION. †THE PRE-REGISTERED TABLE.

m	h	p^*	Jev	OpenJev	Qwen2.5	Phi-3.5	OLMo-2
0.5	0.5	25%	+29	+25	+21	+52	+20
0.5	1	33%	+54	+50	+46	+77	+28
0.5	2	50%	+104	+100	+96	+127	+45
1	0.5	14%	**-2**	**-11**	**-18**	+34	+14
1	1†	18%	+23	+14	+7	+59	+23
1	2	27%	+73	+64	+57	+109	+40
2	0.5	7%	**-69**	**-83**	**-97**	**-2**	+4
2	1	10%	**-44**	**-58**	**-72**	+23	+13
2	2	14%	+6	**-8**	**-22**	+73	+30

VIII. DOES BETTER RANKING REDUCE COST?

We pre-registered the cost test before computing any cost outcome. Each model gets 50 reviews (10% of the held-out bugs), and a review sends the case to a person. The primary arm ranks candidates by the expected saving of a review for their action, estimated on development data. It is compared with confidence-ranked and random review.

Two of ten pre-registered tests were supported: the action rule beat random review for Jev and OpenJev, by about 10–12 cost units. It never beat confidence, and every review arm cost more than no review at all. For Jev that was 658 with no review against 681–714 with review (Fig. 5).

The oracle shows where the money is. For the Jev-style models, 40–52% of no-review cost comes from wrongly accepted serious or regression bugs, and picking the right 50 cases would save 33–44%. Those cases are only 15–16% of accepted cases, and nothing we tested could find them. Among Jev's 245 accepted held-out cases, the decision estimator, final-step confidence and `score`'s top-level probability ranked the 38 wrong accepts at AUROC 0.46, 0.49 and 0.49. For the general models the problem is different: they over-escalate, and even the oracle saves at most 5%.

The conclusion depends on the costs (Table VI). In an exploratory re-scoring within a factor of 2 of the frozen matrix, review pays for Jev, OpenJev and Qwen2.5 when a wrong accept costs twice as much (for Jev, 44–69 units below no review) or a review half as much, because the break-even precision then falls to about the 14–16% share of wrong accepts. The ordering of the arms is steadier than their sign: the action rule beats random for both Jev-style models in 12 of 16 cells. If reviewers are right only 80% of the time, Jev's saving at twice the wrong-accept cost falls from 44 to 26 units but does not disappear. We also tried a per-case score built to find the expensive errors. On the development mix it cut cost

by 12% against no review (630 vs. 713), but the development bugs were quota-sampled, and re-weighted to the natural mix it was projected to cost more than no review. Its pre-registered feasibility check required power of at least 0.80 with at most 1,000 fresh bugs. The highest power was 0.20, so we stopped that study before collecting new data.

IX. Lessons from our own audits

Our audit process caught seven defects, five of them before the data that would have exposed them existed. Among them were a "primitive-aware" strategy that turned out to be identical to the baseline, a trigger that fired on about 90% of inputs, label leaks in constructed content, and a bootstrap seed that made two reported bounds vary between runs by up to 0.005. The confidence-versus-decisions confound was found only after a result. It would have been caught earlier by asking, for each strategy and each target class, whether the strategy had the information it needed to be right. We suggest that check, together with measuring each trigger's selectivity on pilot inputs, before any escalation policy is frozen.

X. Threats to validity

The largest threat is construct validity. Failure means disagreeing with a target built from tracker labels, and those labels are not ground truth: Mozilla's sample excludes re-triaged bugs, kernel severities are reporters' claims, and the kernel's middle level is a default value. In an exploratory pilot, one AI assistant chose actions for 50 held-out bugs and agreed with the target at about chance (Cohen's κ between -0.157 and $+0.015$). With one rater we cannot tell whether the target or the judge is at fault. A two-human validation is designed but not yet run. The cost matrix is also our construct, and real review costs are unknown.

On internal validity, the cost test was pre-registered after the prediction results on the same held-out bugs were known, though before any cost was computed, and the cost-sensitivity and label-shift analyses are exploratory. Old public bugs may appear in model training data, and the recent subsets we can check are small (Mozilla 54, kernel 35). The protocols were revised several times, including a readout bug in the cross-model study fixed after interim results were seen. Every revision is dated and disclosed.

On external validity, the study covers five models in one domain, software issue triage, across two workflows and one Jev version. On Kubernetes the general models were below chance on priority, and the Kubernetes leakage read was done by an AI assistant at our direction.

XI. Conclusion

In a chain of three typed decisions, what the workflow decided predicted its failures far better than how confident it was. This held on 500 new bugs, for five models, and on an independent Kubernetes workflow. Most of the signal is the error rate of the chosen action, and that rate depends on the label mix it was measured under. Predicting failures did not make review worthwhile. The costly errors are wrong accepts that look like correct accepts to every signal we tested, so whether review pays comes down to the ratio of error cost to review cost and to how precisely a signal can find those cases.

XII. Future scope

The first step is to validate the target with two independent human evaluators. Beyond that, the oracle points to the real opportunity: a signal that separates costly errors within a single action. OpenJev's weak within-action severity signal (AUROC 0.68) is a place to start. The study should also be repeated outside software triage, with newer model versions, supervised baselines and measured review costs.

Acknowledgment

The authors used an AI assistant (Claude, Anthropic) throughout this work: to write and run analysis code, to draft and edit the text, and for the analysis tasks disclosed where they occur (the Kubernetes leakage read, the exploratory target-validity pilot and an independent review of the template content). The authors directed the study, approved every pre-registration and every model run, and take full responsibility for the content.

References

[1] T. Deußer, L. Sparrenberg, and R. Sifa, "Evaluating and benchmarking the system one model Jev," *arXiv preprint arXiv:2609.37647*, 2026.

[2] L. Meng, "How far can a commercial decision model's probabilities be trusted? A calibration audit of TypeSafe Jev against open and general-purpose classifiers," Code, data and paper, Zenodo, https://doi.org/10.5281/zenodo.22935043, 2026, version 1.0.0.

[3] S. Denney and M. DiGiuseppe, "JEV versus LLMs: Accuracy, cost and calibration on seven political science replications," *arXiv preprint arXiv:2610.06625*, 2026.

[4] M. Lin, Q. Lan, S. Guan, W. Jing, J. Fan, D. Gregg, and G. Botterweck, "Uncertainty quantification for LLM agents: A taxonomy, an evaluation protocol, and an empirical study," *arXiv preprint arXiv:2609.07395*, 2026.

[5] TypeSafe, "Confidence," TypeSafe documentation, https://docs.typesafe.ai/confidence, 2026, accessed 2026-10-16.

[6] J. Geuter, A. Cheng, and D. Alvarez-Melis, "Releasing the inner Jev: A simple recipe for open decision models," *arXiv preprint arXiv:2610.17205*, 2026.

[7] J. C. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, A. J. Smola, P. Bartlett, B. Schölkopf, and D. Schuurmans, Eds. MIT Press, 1999, pp. 61–74.

[8] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.

[9] R. El-Yaniv and Y. Wiener, "On the foundations of noise-free selective classification," *Journal of Machine Learning Research*, vol. 11, pp. 1605–1641, 2010.

[10] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 4878–4887.

[11] D. Madras, T. Pitassi, and R. Zemel, "Predict responsibly: Improving fairness and accuracy by learning to defer," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[12] H. Mozannar and D. Sontag, "Consistent estimators for learning to defer to an expert," in *Proceedings of the 37th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 119, 2020, pp. 7076–7087.

[13] G. Bansal, B. Nushi, E. Kamar, E. Horvitz, and D. S. Weld, "Is the most accurate AI the best teammate? optimizing AI for teamwork," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 13, 2021, pp. 11 405–11 414.

[14] S. Park, M. Choi, J. Kim, and M. Kang, "Risk is not review value: Wrong-answer exposure under bounded review budgets," in *SeT-LLM Workshop at KDD*, 2026, arXiv:2609.07095.
[15] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002.
[16] Z. C. Lipton, Y.-X. Wang, and A. J. Smola, "Detecting and correcting for label shift with black box predictors," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.
[17] J. Kittler, M. Hatef, R. P. W. Duin, and J. Matas, "On combining classifiers," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 20, no. 3, pp. 226–239, 1998.
[18] V. Kotte, "PASC: Pipeline-aware conformal prediction with joint coverage guarantees for multi-stage NLP and LLM pipelines," *arXiv preprint arXiv:2605.18812*, 2026.
[19] J. Zhang, S. Elluri, B. Cherukuvada, Y. Joffe, J. Sena, M. Contreras, S. Siegel, S. Nerella, C. Price, and P. Rashidi, "Auditing multimodal LLM raters: Central tendency bias in clinical ordinal scoring," *arXiv preprint arXiv:2605.16386*, 2026.
[20] M. J. Zellinger, R. Liu, and M. Thomson, "Cost-saving LLM cascades with early abstention," *arXiv preprint arXiv:2502.09054*, 2025.
[21] D. Bouchard, "Is escalation worth it? On the depth of LLM cascades," *arXiv preprint arXiv:2605.06350*, 2026, v2; v1 was titled "Is Escalation Worth It? A Decision-Theoretic Characterization of LLM Cascades".
[22] J. Jung, F. Brahman, and Y. Choi, "Trust or escalate: LLM judges with provable guarantees for human agreement," in *International Conference on Learning Representations (ICLR)*, 2025.
[23] V. Cortellessa and V. Grassi, "A modeling approach to analyze the impact of error propagation on reliability of component-based systems," in *Component-Based Software Engineering (CBSE 2007)*, ser. LNCS, vol. 4608. Springer, 2007, pp. 140–156.
[24] A. Lamkanfi, S. Demeyer, E. Giger, and B. Goethals, "Predicting the severity of a reported bug," in *7th IEEE Working Conference on Mining Software Repositories (MSR)*, 2010, pp. 1–10.
[25] OpenJev, "OpenJev: an open-weights decision model," https://huggingface.co/openjev/openjev, 2026, Hugging Face commit b4a89f5; CC BY-NC 4.0 weights; states it is independent of TypeSafe. Not the same project as other repositories named OpenJev. Accessed 2026-10-17.
[26] Qwen, A. Yang *et al.*, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.
[27] Team OLMo, P. Walsh, L. Soldaini *et al.*, "2 OLMo 2 furious," *arXiv preprint arXiv:2501.00656*, 2024.
[28] M. Abdin, J. Aneja, H. Awadalla *et al.*, "Phi-3 technical report: A highly capable language model locally on your phone," *arXiv preprint arXiv:2404.14219*, 2024.